- clone() 像 fork() 一样复制文件描述符引用——底层 struct file 和偏移共享，但描述符表独立。这与 Linux 的 CLONE_FILES 不同，是有意保留的简化。

threadtest 创建两个线程，用内核同步对象保护共享计数器，允许它们被调度到不同 CPU，预期输出 threadtest: PASS counter=2000。

8.5 libc 与 POSIX 源码兼容层

8.5.1 libc.a

在最简形态里，用户库被打包成一个静态库 libc.a（也就是 make DYNAMIC_USER=0 时保留的回退路径）：

- ulib.c：字符串、内存函数、gets、stat、atoi；
- usys.S：系统调用桩；
- printf.c：printf/fprintf，支持 %d %x %p %s %c；
- umalloc.c：K&R 风格的 malloc/free，用 sbrk 向内核要内存；
- thread.c：上一节的线程包装；
- posix.c：POSIX 包装层；
- tweetnacl.c、ssh_crypto.c：密码学原语（sshd 使用）。

8.5.2 动态链接 libc.so

上面的静态 libc.a 让每个程序各自携带一份 libc 代码。本项目把默认形态改成共享库：Makefile 先把每个 libc 源文件编成位置无关对象（-fPIC），再用 ld -shared 链成 ET_DYN 的 libc.so，SONAME 取名 libc.so；用户程序则以 -pie 链接到它：

```
$U/%.pic.o: $U/%.c
    $(CC) $(CFLAGS) -fPIC -c -o $@ $<
$U/libc.so: $(LIBC_PIC_OBJS)
    $(LD) $(LDFLAGS) -shared --no-undefined -soname libc.so -o $@ $^
_%: %.o $(ULIB) $(USER_LINK_STAMP)
    $(LD) $(LDFLAGS) -pie --no-dynamic-linker -e main -o $@ ...
```

关键在最后一条：-pie 产出 ET_DYN 可执行文件，--no-dynamic-linker 则不写 PT_INTERP——因为本项目没有用户态 ld.so，动态装载的活由内核 exec() 自己完成。要退回上一节的静态形态，只需 make DYNAMIC_USER=0；构建用 .dynamic-user/.static-user 两个模式戳保证切换时重链所有用户程序，根文件系统构建再把共享库安装到 /lib/libc.so。

exec() 里的装载流程. 对每个 ET_DYN 程序，kernel/exec.c：

1. 校验 ELF 头后按程序头把主镜像映射到虚拟地址 0，并记下 PT_DYNAMIC 位置。为此 loadseg() 放宽了老 xv6 的页对齐要求，只校验 GNU ld 保证的 p_vaddr ≡ p_offset (mod p_align)，以容纳非页对齐的段边界。
2. parsedynamic() 逐条读 DT_* 项，收集 DT_NEEDED、DT_SYMTAB、DT_STRTAB、DT_HASH、DT_RELA/RELASZ/RELAENT 与 DT_JMPREL/PLTRELSZ/PLTREL；符号数取自 ELF hash 表头，动态表必须以 DT_NULL 结束。
3. 策略校验：DT_NEEDED 必须恰好一个、且字符串正是 "libc.so"，随后打开硬编码路径 /lib/libc.so。任意依赖搜索留给未来的用户态 ld.so，不藏进内核。
4. libc 的载入基址取 libbase = PGROUNDUP(sz)，即紧跟主镜像之后的第一个空闲页，loadimage() 据此把 libc.so 也映射进本页表。